Техническое задание на разработку `getmilk.py`

1. Общее описание

Разработать REST API для управления задачами (to-do) на Python с использованием фреймворка Flask. Приложение хранит задачи в памяти процесса, поддерживает полный набор CRUD-операций, фильтрацию, поиск, сортировку, пагинацию и контроль дубликатов.

Стек: Python 3.9+, Flask 3.x.
Формат данных: JSON (`application/json`).
Хранение: в памяти (список Python), без базы данных.
Аутентификация: отсутствует.

2. Модель данных

Задача (`task`) — JSON-объект со следующими полями:

- `id` — integer, генерируется автоматически, уникальный, монотонно возрастает начиная с 1;
- `title` — string, обязательный, непустой, не длиннее 200 символов, уникальный без учёта

регистра и лишних пробелов;
- `done` — boolean, необязательный при создании, по умолчанию `false`;
- `created_at` — string в формате ISO 8601 (UTC), генерируется автоматически, например `2026-10-19T10:18:26.123456+00:00`.

Нормализация `title` для сравнения: схлопнуть все пробельные последовательности в один пробел и привести к нижнему регистру.

3. Начальные данные (seed)

При старте приложения хранилище заполняется пятью демонстрационными задачами:

- `Купить молоко` — `done: true`, создана 5 часов назад;
- `Выучить REST` — `done: false`, создана 4 часа назад;
- `Сдать лабораторную` — `done: true`, создана 3 часа назад;
- `Проверить почту` — `done: false`, создана 2 часа назад;
- `Позвонить маме` — `done: false`, создана 1 час назад.

Идентификаторы — от 1 до 5. После заполнения `next_id` = 6.

4. Эндпоинты

4.1. `GET /` — описание сервиса

Возвращает `200` и JSON-объект с полями `service` (строка `"tasks-api"`), `seed` (текущее количество задач, чтобы быстро убедиться, что сервер жив и данные на месте) и `endpoints` (массив строк с кратким описанием маршрутов).

4.2. `GET /tasks` — список с фильтрами, сортировкой, пагинацией

Поддерживает query-параметры:

- `done` — `true` или `false`, фильтр по статусу; любое другое значение → `400`;
- `q` — строка поиска, ищется подстрока в `title` регистронезависимо с нормализацией;
- `sort` — одно из `id`, `title`, `created_at`, по умолчанию `created_at`; иное значение → `400`;
- `order` — `asc` или `desc`, по умолчанию `asc`; иное значение → `400`;

- `page` — целое ≥ 1, по умолчанию `1`; меньше 1 или не целое → `400`;
- `per_page` — целое от 1 до 100, по умолчанию `10`; вне диапазона или не целое → `400`.

Ответ `200` — JSON-объект с полями `items` (массив задач текущей страницы), `page`, `per_page`, `total` (всего задач после фильтрации), `pages` (количество страниц, минимум 1).

4.3. `GET /tasks/<id>` — одна задача

Возвращает `200` и объект задачи. Если задачи нет — `404` с телом `{"error": "задача не найдена"}`. Если `<id>` не целое — Flask вернёт `404` до обработчика (поведение конвертера `<int:>`, реализовывать не нужно).

4.4. `POST /tasks` — создание

Тело запроса — JSON-объект с полями `title` (обязательно) и `done` (необязательно). Если тело не JSON-объект (невалидный JSON, массив, число, `null`) — `400`.

Валидация `title`:

- поле отсутствует → `400`;

- не строка или пустая после `strip()` → `422`;
- длина после `strip()` больше 200 → `422`;
- совпадает с существующим `title` (с нормализацией) → `409`.

Ответ `201` — созданная задача.

4.5. `PUT /tasks/<id>` — полная замена

Поведение как у `POST`, но: если задачи нет — `404`; `title` проверяется с исключением собственного id (своё прежнее название дубликатом не считается); при отсутствии `done` в теле он сбрасывается в `false` (полная замена). Ответ `200` — обновлённая задача.

4.6. `PATCH /tasks/<id>` — частичное обновление

Обновляются только переданные поля. Для `title`: валидация как в `POST`, но с исключением собственного id; если новое название после нормализации совпадает с текущим — проверка на дубликат не выполняется. Для `done`: приводится к `bool`. Ответ `200` — обновлённая задача. Ошибки: `404` — нет задачи; `400` — тело не JSON-объект; `422` — невалидный `title`; `409` — дубликат.

4.7. `DELETE /tasks/<id>` — удаление

Ответ `204` с пустым телом. Если задачи нет (в том числе при повторном DELETE) — `404`.
Идемпотентность реализуется через `404`, а не через повторный `204`.

5. Общие требования

5.1. Формат ошибок

Все ошибки — JSON-объект вида `{"error": "текст сообщения на русском"}`.

5.2. Коды состояния

- `200` — успешное чтение или обновление;
- `201` — создание;
- `204` — удаление (тело пустое);
- `400` — некорректный запрос (тело не JSON-объект, невалидный query-параметр);
- `404` — ресурс не найден;
- `409` — конфликт (дубликат `title`);
- `422` — семантически невалидные данные (`title` пустой или слишком длинный).

5.3. Вспомогательные функции

- `now_iso()` — текущее время в ISO 8601 UTC;

- `normalize(title)` — нормализация для сравнения (пробелы + регистр);
- `err(message, code)` — формирование JSON-ответа об ошибке;
- `find_task(task_id)` — поиск задачи по id или `None`;
- `title_exists(title, exclude_id=None)` — проверка дубликата;
- `validate_title(data, exclude_id=None)` — общая валидация `title`; возвращает `(title, None)` или `(None, error_response)`;
- `seed()` — заполнение начальными данными при старте.

5.4. Константы

`MAX_TITLE_LEN = 200`, `DEFAULT_PER_PAGE = 10`, `MAX_PER_PAGE = 100`.

6. Шаблон

Ниже — скелет `app.py` со всеми маршрутами, но с пустыми телами функций и `TODO`-комментариями. Заполните их по ТЗ выше.

```
"""Tasks API — CRUD задач на Flask.
```

```
Реализуйте функции по техническому заданию.
Все маршруты уже объявлены, тела функций пусты.
"""

from flask import Flask, request, jsonify
from datetime import datetime, timezone,
timedelta

app = Flask(__name__)

tasks = []
next_id = 1

MAX_TITLE_LEN = 200
DEFAULT_PER_PAGE = 10
MAX_PER_PAGE = 100



# ----------------------------------------
--------------------------------
# Вспомогательные функции
# ----------------------------------------
--------------------------------

def now_iso():
    """Текущее время в ISO 8601 UTC."""
    # TODO: вернуть
datetime.now(timezone.utc).isoformat()
```

```
    raise NotImplementedError


def normalize(title):
    """Нормализация title для сравнения:
схлопнуть пробелы + нижний регистр."""
    # TODO: " ".join(title.split()).lower()
    raise NotImplementedError


def err(message, code):
    """JSON-ответ об ошибке: {"error": message}
с заданным кодом."""
    # TODO: return jsonify({"error": message}),
code
    raise NotImplementedError


def find_task(task_id):
    """Найти задачу по id или вернуть None."""
    # TODO: линейный поиск по tasks
    raise NotImplementedError


def title_exists(title, exclude_id=None):
    """Есть ли уже задача с таким title (с
нормализацией)?
```

```
    exclude_id — id задачи, которую
переименовываем (её собственное название
    дубликатом не считается).
    """
    # TODO: any(...) по tasks с учётом
exclude_id и normalize
    raise NotImplementedError


def validate_title(data, exclude_id=None):
    """Валидация title. Возвращает (title,
None) или (None, error_response).

    Правила:
      - отсутствует           -> 400
      - не строка / пустая    -> 422
      - длиннее MAX_TITLE_LEN -> 422
      - дубликат              -> 409
    """
    # TODO: реализовать по шагам, каждый шаг —
return None, err(...)
    raise NotImplementedError


def seed():
    """Заполнить хранилище демонстрационными
задачами при старте."""
    # TODO: 5 задач с id 1..5, created_at = now
```

```
- 5ч + i часов, next_id = 6
    raise NotImplementedError


# Вызов seed() при импорте модуля
# seed()


# ----------------------------------------------------------------
# Вьюхи
# ----------------------------------------------------------------

@app.get("/tasks")
def list_tasks():
    """GET /tasks — список с фильтрами, сортировкой и пагинацией.

    Query: done, q, sort, order, page, per_page.
    Ответ: {items, page, per_page, total, pages}.
    """
    # TODO: реализовать по ТЗ п. 4.2
    raise NotImplementedError
```

```python
@app.get("/tasks/<int:task_id>")
def get_task(task_id):
    """GET /tasks/<id> — одна задача. 404, если нет."""
    # TODO: find_task -> 404 или jsonify(task), 200
    raise NotImplementedError


@app.post("/tasks")
def create_task():
    """POST /tasks — создать задачу.

    Тело — JSON-объект {title, done?}.
    Валидация title через validate_title().
    Ответ 201 с созданной задачей.
    """
    # TODO: реализовать по ТЗ п. 4.4
    raise NotImplementedError


@app.put("/tasks/<int:task_id>")
def replace_task(task_id):
    """PUT /tasks/<id> — полная замена ресурса.

    Как POST, но: 404 если нет задачи; validate_title(..., exclude_id=task_id);
    отсутствующий done -> False.
```

```
    """
    # TODO: реализовать по ТЗ п. 4.5
    raise NotImplementedError


@app.patch("/tasks/<int:task_id>")
def update_task(task_id):
    """PATCH /tasks/<id> — частичное
обновление.

    Обновляются только переданные поля (title
и/или done).
    """
    # TODO: реализовать по ТЗ п. 4.6
    raise NotImplementedError


@app.delete("/tasks/<int:task_id>")
def delete_task(task_id):
    """DELETE /tasks/<id> — удалить задачу. 204
или 404."""
    # TODO: реализовать по ТЗ п. 4.7
    raise NotImplementedError


@app.get("/")
def index():
    """GET / — описание сервиса."""
```

```
    # TODO: {"service": "tasks-api", "seed": len(tasks), "endpoints": [...]}
    raise NotImplementedError


if __name__ == "__main__":
    app.run(debug=True)
```

7. Критерии готовности

Все эндпоинты возвращают коды и тела, описанные в разделе 4. Поле `title` валидируется одинаково в `POST`, `PUT`, `PATCH` (через общую `validate_title` или эквивалентную логику). Дубликаты `title` дают `409` с учётом нормализации; при `PUT` и `PATCH` собственное название дубликатом не считается. `GET /tasks` поддерживает все шесть query-параметров и возвращает объект `{items, page, per_page, total, pages}`. `DELETE` возвращает `204` с пустым телом, повторный — `404`. `GET /` возвращает `service`, `seed`, `endpoints`. При старте в хранилище 5 задач, `next_id = 6`.